

CS240 Algorithm Design and Analysis
Spring 2026
Problem Set 4

Due: 23:59, June 10, 2026

1. Please write your solutions in English.
2. Submit your solutions to **Gradescope**. When submitting, match your solutions to the problems correctly.
3. If you want to submit a handwritten version, scan it clearly.
4. Up to 2 late days are allowed for homework submissions. Assignments submitted within the first 24 hours after the deadline will incur a 10% penalty; those submitted within 48 hours will incur a 20% penalty. Submissions more than 48 hours late will not be accepted unless there is a valid reason, such as a medical or family emergency.
5. You are required to follow ShanghaiTech's academic honesty policies. You are allowed to discuss problems with other students, but you must write up your solutions by yourselves. You are not allowed to copy materials from other students or from online or published resources. Violating academic honesty can result in serious penalties.

Problem 1: Lazy Disk-Write Queue

A cloud storage system manages n data pages, numbered $1, 2, \dots, n$. After a page is modified, it becomes a *dirty page*, meaning that the page has unsaved contents that have not yet been written back to disk.

The system maintains a Boolean array `dirty` $[1 \dots n]$, where `dirty` $[i] = \text{true}$ means that page i has unsaved modifications. The system also maintains a FIFO queue Q , whose entries are page IDs requested for write-back.

The system supports the following two operations:

- `MODIFY`(i): set `dirty` $[i] \leftarrow \text{true}$, and append i to the rear of Q . Even if `dirty` $[i]$ is already `true`, this operation still appends another copy of i to Q .
- `FLUSHONE`(): repeatedly remove the front entry j from Q . If `dirty` $[j] = \text{false}$, then this queue entry is stale and is discarded. If `dirty` $[j] = \text{true}$, then page j is written back, `dirty` $[j] \leftarrow \text{false}$, and j is returned. If Q becomes empty, the operation returns 0.

Each queue insertion, queue deletion, array access, and array assignment has cost 1. Writing back one dirty page also has cost 1.

Answer the following questions:

- (a) Construct an operation sequence in which one call to `FLUSHONE`() has cost $\Theta(m)$, where m is the number of operations performed so far.
- (b) Prove that the total running time of any sequence of m operations is $O(m)$. Your proof should also explain why duplicate page IDs in Q do not break the bound.
- (c) Give a potential function and use the potential method to prove that both `MODIFY` and `FLUSHONE` have amortized running time $O(1)$.

Problem 2: Triangular Calibration Monitor

A scientific monitoring device continuously receives samples. The operation `INSERTSAMPLE( $x$ )` inserts one new sample and has base cost 1. Let N be the total number of samples inserted so far. Initially, $N = 0$.

Whenever N becomes a triangular number, the device automatically performs one calibration. Define $T_0 = 0$, and for $q \geq 1$,

$$T_q = 1 + 2 + \cdots + q = \frac{q(q+1)}{2}.$$

When the q -th calibration is triggered, its cost is exactly q . For example, after the 1st insertion the calibration cost is 1, after the 3rd insertion the calibration cost is 2, after the 6th insertion the calibration cost is 3, and after the 10th insertion the calibration cost is 4.

Answer the following questions:

- (a) Given n , how many calibrations occur during the first n insertions?
- (b) Use aggregate analysis to prove that the total cost of the first n insertions is $O(n)$.
- (c) Use the accounting method to give a constant amortized charge for each insertion.

Problem 3: Archive with Tombstone Compaction

An online news platform stores articles in an array. Each array entry is either a live article or a tombstone. A tombstone means that the article has been deleted, but its array position has not yet been physically removed.

The system also maintains a mapping from article IDs to array positions. Therefore, checking whether an ID corresponds to a live article and marking it as a tombstone can both be done in $O(1)$ time. During compaction, the system also updates this mapping. Updating the mapping for a copied live article is included in the copying cost of that live article.

Users can perform the following two external operations:

- **ADD(*article*)**: append the new article as a live article at the end of the array, and record the mapping from its ID to its array position.
- **REMOVE(*id*)**: if the article with ID *id* is still live, mark its entry as a tombstone. If this ID does not exist or is no longer live, perform only one lookup and return.

The system also has the following internal cleanup procedure:

- **COMPACT()**: scan the entire array, copy all live articles into a new array, discard all tombstones, and update the ID-to-position mapping.

COMPACT() is not directly called by users. It is automatically triggered by the system after a **REMOVE** operation.

System rule: after each **REMOVE**, if the number of tombstones is strictly greater than the number of live articles, the system automatically executes **COMPACT()**.

Assume that appending an article costs 1, marking a tombstone costs 1, and scanning or copying one array entry costs 1.

Answer the following questions:

- (a) Explain why a single **REMOVE** operation can have worst-case cost $\Theta(n)$, where n is the array length immediately before the **COMPACT** operation triggered by this **REMOVE**.
- (b) Prove that the total cost of any m external operations is $O(m)$.
- (c) Give a potential function based on the number of tombstones and prove an amortized $O(1)$ bound.

Problem 4: Random Audit without Replacement

A course has n homework submissions, and exactly r of them have formatting violations. The instructor randomly selects s distinct submissions for manual audit, using sampling without replacement. Assume $0 \leq r \leq n$ and $0 \leq s \leq n$.

Let X be the number of selected submissions that have formatting violations.

Answer the following questions:

- (a) Define suitable indicator random variables and compute $\mathbb{E}[X]$.
- (b) Compute the probability that the audit finds no formatting violation.
- (c) Give a simple exponential upper bound for the probability in part (b), and use it to give a sufficient condition on s so that the instructor finds at least one violation with probability at least $1 - \delta$, where $r > 0$ and $0 < \delta < 1$.

Problem 5: One-Round Random-Priority Conflict Filter

A system has a set of tasks. Some pairs of tasks conflict and cannot be selected simultaneously. This conflict relation is represented by a simple undirected graph $G = (V, E)$: each vertex is a task, and each edge means that the two corresponding tasks conflict.

The system performs the following randomized filtering step:

1. Each vertex v independently chooses a priority $p(v)$ uniformly at random from $[0, 1]$.
2. Vertex v is selected if its priority is smaller than the priority of every neighbor of v . Here a smaller numerical priority means a higher priority.

Because the priorities are drawn from a continuous distribution, all priorities are distinct with probability 1. Let S be the set of selected vertices.

Answer the following questions:

- (a) Prove that S is always an independent set.
- (b) For a fixed vertex v , let $\deg(v)$ be its degree. What is the probability that v is selected?
- (c) Let $X = |S|$. Use indicator random variables to compute $\mathbb{E}[X]$, and simplify the answer when G is d -regular.